

com/invite in a web browser.

1. In the Invitation Code field, enter the invitation code provided by your instructor or in the LevelUp LMS.

1. Select Provision Training Environment.

1. For self-paced courses, you will need your GitLab.com username. To find your username, navigate to Gitlab.

1. In the left sidebar, select your profile image.

1. In the resulting dropdown, you will see a value starting with @. This is your GitLab username.

1. After inputting your invitation code, the portal prompts you for your GitLab.com username. Enter your GitLab.com user (excluding the leading @ sign) in the field provided.

1. Select Provision Training Environment.

1. Select My Group at the bottom of the page.

1. Sign in with your GitLab.com credentials.

1. You will be redirected to a My Test Group group that provides a sandbox for you to perform training lab steps in.

> This group has a GitLab Ultimate license to see all of the features while your personal username namespace requires a paid subscription or a free trial to access all of the features.

>

> If you receive a 404 error when accessing your group, it is likely due to the username input during the lab provisioning. Double check your GitLab username to make sure it is entered correctly.

1. From your My Test Group training subgroup, click the New project button.

Task B. Create a Project

1. Click the Create blank project tile.

1. In the Project name field, enter Security Labs.

> The project slug will automatically populate. You can change this to a shorter string if desired for your own project. Leave it at the default for this lab.

1. If your lab environment URL starts with <https://gitlab.com/gitlab-learn-labs/>, in the Project URL field, click the dropdown for the second half of the URL to make sure it's pointing to a group name (starts with gitlab-learn-labs/) and not a username. You should create this project inside a group, not directly in your user's namespace.

1. If your group URL starts with `https://ilt.gitlabtraining.cloud/...`, in the Project URL field, click the dropdown for the second half of the URL to make sure it's pointing to a group name (starts with `training-users/`) and not a username. You should create this project inside a group, not directly in your user's namespace.

1. Under Visibility Level, click Private.

> Since the parent group above your group is private, all child groups and projects below will be private. You can learn more about project visibility levels in the documentation.

1. Check Initialize repository with a README.

1. Ensure that the Enable Static Application Security Testing (SAST) checkbox remains unchecked. For this lab, we will add the SAST functionality manually.

1. Click Create project.

Task C. Enable and Configure SAST

> Static Application Security Testing, or SAST, is the process of examining source code for vulnerabilities. You can use a SAST scan to automatically scan a code repository for known vulnerabilities. You can also use a SAST scan to check merge requests for vulnerabilities before merging the request. This process helps ensure that your code stays vulnerability free.

1. Create a new file in the main branch by clicking (+) > This directory > New file.

1. In the Filename field, type `.gitlab-ci.yml`. You will see a template appear automatically that you can leave blank for this task.

1. Define a single test stage:

```
yml
stages:
- test
```

> Keep in mind that YAML files should be indented with two spaces. Your web IDE may try to use a tab with 4 spaces. Simply use the backspace to set 2 spaces if you are not copying and pasting the examples.

1. Enable SAST by pasting the following text after the stages definition in `.gitlab-ci.yml`:

```
yml
include:
- template: Security/SAST.gitlab-ci.yml
```

> It is also possible to configure SAST through the GitLab UI by navigating to Secure > Security configuration and clicking the Enable SAST button. We will be configuring it by editing the CI file for this lab to help you learn more about how it works under the hood.

1. Add a variables section to the end of your .gitlab-ci.yml file and set the SASTEXCLUDEDPATHS: venv/.

```
yml
variables:
  SASTEXCLUDEDPATHS: venv/
```

> You can customize your SAST by adding configurations to the variables section of the .gitlab-ci.yml file. For example, the SASTEXCLUDEDPATHS variable can exclude project paths from the SAST scan. This option can be set to prevent unnecessary scanning of files.

>

> As an example, Python projects often contain a venv directory that contains packages used by the project. Since this directory does not contain our own source code, we should exclude it from the SAST scan.

>

> A full list of SAST variables can be found in the documentation.

1. Click the Commit changes button, add an appropriate commit message (ex. Add SAST template to .gitlab-ci.yml), and ensure that the 'Commit to the current main branch' option is selected.

1. Click the Commit changes button.

1. Once complete, you will have a .gitlab-ci.yml file that looks like this:

```
yml
stages:
- test

include:
- template: Security/SAST.gitlab-ci.yml

variables:
  SASTEXCLUDEDPATHS: venv/
```

Task D. Add a Vulnerability to the Application

> With SAST scanning in place, you are now able to see security vulnerabilities inside of merge requests. To demonstrate how this SAST scan works, let's create a branch with some vulnerable code.

1. Navigate to Code > Repository to see your project source code.

1. Add a new file by clicking (+) > This directory > New file.

1. Set the filename to main.py.

1. Introduce a new vulnerability (pretend this is accidental!) by adding the following code to your main.py file:

```
python
import subprocess

in = input("Enter your server ip: ")
subprocess.run(["ping", in])

gitlabtoken = 'glpat-hC5G3PrMaZ7UkVZhxnX'
rsadata = ""
-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4,ENCRYPTED
DEK-Info: DES-EDE3-CBC,86C3F4011519BFBB
PxyzMAIAmEu/Qkx9nPh696SU7/MjXpCpOnfFijLhJumNcRIWgsOil9rfwlkh4aN
-----END RSA PRIVATE KEY-----
"""

print("Attempting to connect to the server")
print("Application authentication was successful")
```

1. Click the Commit changes button, Add an appropriate commit message (Ex. Add prompt for server authentication), set the target branch to addauth.

> Setting the target branch to addauth will create a new branch named addauth, and open a merge request on the branch.

1. Make sure that the Create a new merge request for this change checkbox is checked.

1. Click the Commit changes button.

1. In the resulting merge request, set the name to Add prompt for server authentication, and leave the rest of the fields as default.

1. Click the Create merge request button.

1. On the merge request page, you will see your security pipeline running. Wait for the pipeline to complete.

1. Once the pipeline completes, a new section will appear in the merge request, showing the results of the security scan.

> You may need to refresh the page to see the new security scan section.

1. In this example, the security scan will show 1 new high vulnerability. To view the details of the vulnerability report, click the View all pipeline findings option.

Task E. Merge Request Vulnerability Report

> One of the main goals of security scanning is to prevent insecure code from making it into a repository. You can use the merge request vulnerability report to see all of the vulnerabilities that were detected in a single merge request. Note that this report will only

show vulnerabilities that are newly introduced in the current merge request. If a vulnerability already exists in the repository, it will not show here, but will show in the project level vulnerability report.

1. In the vulnerability column, click the Improper neutralization of special elements used in an OS Command ('OS Command Injection') vulnerability.

1. In the resulting window, you will see details about the OS Command Injection vulnerability. Note that the detection shows the location of the vulnerability, which is line 4 of main.py. Click the X in the top right to close the modal window, or the Cancel button at the bottom.

1. In the sidebar, navigate to Code > Merge requests.

1. Click the merge request Add prompt for server authentication.

1. Click the Changes tab below the merge request title. On the main.py file, click vertical ellipses -> Edit in single-file editor.

1. The scan showed that line 4 contained a vulnerability.

```
python
main.py Line 4
subprocess.run(["ping", in])
```

> The problem is that this line uses user input to run a system command. In this situation, a user could craft an input that causes the application to execute an unintended command. To resolve this, you can either remove the command, or remove the user input from the command.

1. Remove lines 1 through 4 from the code. This is what the file should look like now.

```
python
gitlabtoken = 'glpat-hC5G3PrMaZ7UkVZhxnX'
rsadata = ""
-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4,ENCRYPTED
DEK-Info: DES-EDE3-CBC,86C3F4011519BFBB
PxyzMAIAmEu/Qkx9nPh696SU7/MjXpCpOnfFijLhJumNcRIWgsOil9rfwlkh4aN
-----END RSA PRIVATE KEY-----
"""

print("Attempting to connect to the server")
print("Application authentication was successful")
```

1. Click the Commit changes button, then click the Commit changes again.

> You may need to refresh the page to see the latest changes.

1. Click the Overview tab below the merge request title.

> In the overview, you will see the security pipeline running. When the pipeline completes, refresh the page to see the security report. The security report will show no new potential vulnerabilities detected. With no new vulnerabilities detected, the branch is now safe to merge into main!

1. Leave Delete source branch checked and click Merge.

Task F. Enable and Configure Secret Detection

> In the last section, you applied SAST to detect vulnerabilities in your source code. In addition to scanning code for vulnerabilities, GitLab can also scan your code for secrets like keys and API tokens. Adding secret detection to your code will prevent leaking sensitive data in your repositories.

>

> The Secret Detection job belongs to the test stage by default. Since your `.gitlab-ci.yml` already defines that stage, you don't need to define it again.

1. In the Left sidebar, navigate to Code > Repository.

1. Click the `.gitlab-ci.yml` file. In the top right above the code, navigate to Edit > Edit single file.

1. Enable Secret Detection by adding template: `Security/Secret-Detection.gitlab-ci.yml` at the end of the existing `include:` section in `.gitlab-ci.yml`, below the template for SAST. This indent should be at the same level as the previous template.

```
yml
include:
- template: Security/SAST.gitlab-ci.yml
- template: Security/Secret-Detection.gitlab-ci.yml
```

> It is also possible to configure Secret Detection through the GitLab UI by navigating to Secure > Security configuration and clicking the Configure with a merge request button under Pipeline Secret Detection. We will be configuring it by editing the CI file for this lab to help you learn more about how it works under the hood.

1. Configure Secret Detection to ignore the test directory by pasting this job definition at the end of `.gitlab-ci.yml`. The first line should have no indent.

```
yml
secretDetection:
  variables:
    SECRETDETECTIONEXCLUDEDPATHS: tests/
```

> To configure Secret Detection to use non-default behavior, you can override the `secretDetection` job definition and add variables inside it.

>

> A full list of Secret Detection variables can be found in the documentation.

1. Your .gitlab-ci.yml file will now look like this.

```
yml
stages:
- test

include:
- template: Security/SAST.gitlab-ci.yml
- template: Security/Secret-Detection.gitlab-ci.yml

variables:
  SASTEXCLUDEDPATHS: venv/

secretDetection:
  variables:
    SECRETDETECTIONEXCLUDEDPATHS: tests/
```

1. You have already learned how to commit your changes to a new branch and create a merge request. Commit your changes to the addsecretDetection target branch. The commit message can be left at default or updated to Add Secret Detection to .gitlab-ci.yml. For single commit branches, the commit message is used as the merge request title.

> If you look at the security report on this merge request, you will notice that no vulnerabilities have been detected. This occurs because the secrets in main.py already exist in the main branch. The scan that occurs in the merge request will only show vulnerabilities that are newly introduced in the merge request. To see existing vulnerabilities, you will need to look at the project level vulnerability report covered in the next section.

1. Click the Merge button on your merge request after the pipeline passes.

Task G. View the Project Level Vulnerability Report

> Every time you merge code into the main branch, the security pipeline will run and generate a project level vulnerability report that shows all vulnerabilities in the latest commit to the default branch. Think of this as the baseline set of vulnerabilities that you'll compare to vulnerabilities on other branches.

1. Navigate to Secure > Vulnerability Report.

1. Looking at the Tool column in the Development vulnerabilities table, you'll see two critical vulnerabilities detected by the Secret Detection scan you just added.

1. Click on some of vulnerabilities to learn more about them, and to see where they occur in the code.

1. Experiment with the Status, Severity, and Tool filters.

Task H. Confirm the Vulnerability

> When the security scanners find vulnerabilities, you need to keep track of whether they should be fixed or ignored. You do this by setting a vulnerability's status. There are several ways to do so, but in this lab you'll set status inside the Vulnerability Report.

1. There is a vulnerability in the report with the description RSA private key. This vulnerability indicates that a RSA private key is written in our repository. By looking at the code, we can confirm that there is an RSA private key in the code. The second vulnerability, GitLab Personal Access Token implies that a GitLab token is present in the code. By looking at the code, we can confirm that this is true as well.

1. Since the vulnerabilities do exist in the code, click the checkbox next to each vulnerability. In the Set status dropdown, click Confirm. Click Change status.

Task I. Create a Branch and Merge Request

> You'll need a branch and an MR to fix the RSA private key vulnerability.

1. Navigate to Code > Branches.

1. Click the New branch button.

1. Name the branch fix-rsa.

1. Leave the Create from dropdown as the default value main, and click the Create branch button.

1. You'll be returned to the Repository page for the fix-rsa branch. Click the file main.py.

1. In the top right above the code, navigate to Edit > Edit single file.

1. The code currently looks like this.

```
python
gitlabtoken = 'glpat-hC5G3PrMaZ7UkVZhxnX'
rsadata = ""
-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4,ENCRYPTED
DEK-Info: DES-EDE3-CBC,86C3F4011519BFBB
PxyzMAIAmEu/Qkx9nPh696SU7/MjXpCpOnfFijLhJumNcRIWgsOil9rfwlkh4aN
-----END RSA PRIVATE KEY-----
"""

print("Attempting to connect to the server")
print("Application authentication was successful")
```

1. Remove the first two variables to get the following result.

```
python
```



```
print("Attempting to connect to the server")
print("Application authentication was successful")
```

> In this example, we remove the token and API key. If these values are needed for the code to function, they are often stored as environment variables (or CI variables) rather than written into the code. Doing this helps prevent leaking this data to anyone who can view the code.

1. Click the Commit changes button.

1. Make sure the Commit to the current fix-rsa branch option is selected, then click Commit changes.

1. Click the Create merge request button.

1. Keep all the options as their default, and click Create merge request.

1. On the merge request page, wait for the pipeline to complete.

1. Once the pipeline completes, the security scan will display in the merge request. Click the View all pipeline findings link.

> You may need to refresh the page for the security scan to show.

1. Examine the report. You should no longer see any vulnerabilities.

> Although we have removed the keys in this merge request, they may still exist in the repository history. It is always advised to rotate keys if they are ever detected in a repository.

Task J. Merge your branch and resolve the fixed vulnerability

1. In the merge request, click the Merge button.

1. Navigate to Build > Pipelines.

1. Click the status icon of the pipeline that triggered when you merged your branch.

1. Once all the jobs have finished, navigate to Secure > Vulnerability Report.

1. Click the checkbox next to the RSA private key and GitLab Personal Access Token vulnerability. In the Set status dropdown, click Resolve. Click Change status.

Task K. Enable and Configure DAST

> Dynamic Application Security Testing, or DAST, is the process of scanning a running application for vulnerabilities using simulated attacks. DAST allows you to see how your application actually runs, catching vulnerabilities that may not be present in static testing. In this section, you will learn how to configure DAST scanning in your project.

1. Navigate to Code > Repository.

1. Click on .gitlab-ci.yml.

1. Click Edit > Edit single file.

1. Since the default DAST job belongs to the dast stage, you need to define that stage by pasting this line at the end of the existing stages: section. The dast stage should be at the same indent level as the test stage.

```
yml
stages:
- test
- dast
```

1. Enable DAST by adding template: DAST.gitlab-ci.yml to the end of the existing include: section in .gitlab-ci.yml. This line should have the same indentation as the other templates.

```
yml
include:
- template: Security/SAST.gitlab-ci.yml
- template: Security/Secret-Detection.gitlab-ci.yml
- template: DAST.gitlab-ci.yml
```

1. Add DASTWEBSITE: https://example.com at the end of the existing global variables: section (not the variables: section inside the secretdetection job definition). Use the same indentation as the SASTEXCLUDEDPATHS variable.

```
yml
variables:
  SASTEXCLUDEDPATHS: venv/
  DASTWEBSITE: https://example.com
```

> Normally you would run DAST against your project's code running in either a review environment or a production environment. Since the code in this project is just a single Python file instead of a deployable web app, you'll configure DAST to scan an outside web app that has nothing to do with the code in this project.

>

> A full list of DAST variables can be found in the [documentation](https://docs.gitlab.com/ee/user/applicationsecurity/dast/browser/index.html#available-cicd-variables).

1. After these changes, your .gitlab-ci.yml file should look like this.

```
yml
stages:
- test
```

- dast

include:

- template: Security/SAST.gitlab-ci.yml
- template: Security/Secret-Detection.gitlab-ci.yml
- template: DAST.gitlab-ci.yml

variables:

SASTEXCLUDEDPATHS: venv/
DASTWEBSITE: https://example.com

secretDetection:

variables:
SECRETDETECTIONEXCLUDEDPATHS: tests/

1. Click the Commit changes button.

1. To view the progress of the DAST scan, navigate to Build > Pipelines. Click on the Status of the job to see each individual job in the pipeline. Click dast to see the CI output of the job.

> The DAST scan will take approximately 90 seconds to complete.

1. Once the scan completes, navigate to Secure > Vulnerability Report.

1. In the vulnerability report, note the new vulnerabilities discovered by DAST. Click on each vulnerability to learn more.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab Security Essentials Hands-On Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/secessentialshandson2.md

> Estimated time to complete: 15 minutes

Objectives

Many projects depend on open source software and libraries. The dependencies that a project uses can also contain security vulnerabilities.

In this lab, you will learn how to scan for vulnerabilities in your project dependencies.

Prerequisites

Before beginning this lab and all later labs, you should remove any scanners that you enabled in previous labs to speed up pipeline runtime.

1. Open your browser to the Security Labs project that you created in Lab 1.
2. Remove the lines that contain the DAST stage, and the DAST scanner. It should look similar to the example below.

```
yaml
stages:
- test

include:
- template: Security/SAST.gitlab-ci.yml
- template: Security/Secret-Detection.gitlab-ci.yml

variables:
  SASTEXCLUDEDPATHS: venv/

secretDetection:
  variables:
    SECRETDETECTIONEXCLUDEDPATHS: tests/
```

> It is also possible to disable these features by setting CI variables on the project, such as `DASTDISABLED=true` and `SECRETDETECTIONDISABLED=true`. Learn more in the documentation.

Task A. Add Project Dependencies

> Python-based projects often use pip for package management. With pip, developers will often provide a list of dependencies in a file called `requirements.txt`. Although this example uses Python with the pip package manager, GitLab supports various other package managers. For a full list of supported languages and package managers, [click here](#).

1. In your Security Labs project, navigate to Code > Repository.
1. Create a file by clicking + > This directory > New file
1. Set the filename to `requirements.txt` and add the following text.

```
text
requests==2.27.1
```

1. Enter an appropriate commit message (ex. Adding requirements file), set the Target Branch to main, and click the Commit changes button.

Task B. Enable Dependency Scanning

> The Dependency Scanner will traverse your project dependencies looking for security vulnerabilities. You'll enable Dependency Scanning by including a GitLab-provided template in your CI/CD configuration file. You can do this manually, or you can use the GitLab GUI to make a merge request that does it for you. Since you used the manual technique to enable SAST, Secret Detection, and DAST in the last lab, use the GUI to enable Dependency Scanning in this lab.

1. Navigate to Secure > Security configuration.

1. In the Dependency Scanning pane, click Configure with a merge request. This does 3 things for you:

1. Creates a new branch
2. Adds a commit to the branch which edits your CI/CD configuration file to enable Dependency Scanning
3. Redirects you to a page for creating an MR for that branch

1. On the merge request (MR) creation page, leave all fields at their default values and click the Create merge request button. You will be redirected to the details page for the MR that you just created.

1. In the middle of the page, find the notification that a pipeline is running on the MR's branch.

It could take a few minutes for the pipeline to finish, even if you've disabled scans from the previous lab.

Do not click Auto-merge button since that can lead to unexpected behavior.

Wait for the pipeline to finish. You can watch it by navigating to Build > Pipelines or clicking on the pipeline number in the merge request.

1. After the pipeline has finished, click the Merge button in the MR. You might need to refresh the page to see this button.

1. Navigate to Code > Repository and open .gitlab-ci.yml. Notice that the MR has added documentation at the top, reformatted the file, and added a Dependency Scanning template in the include: section.

Task D. Configure Dependency Scanning

> You can configure Dependency Scanning by setting variables in your the gitlab-ci.yml file. In this task, you'll change the Dependency Scanner's log level to info.

1. Navigate to Build > Pipeline editor.

1. Add this job definition to the bottom of your .gitlab-ci.yml file. It overrides an existing job that's defined by the Dependency Scanning template. Note that there are 2 hyphens and 1 underscore in the first line.

```
yml
gemnasium-python-dependencyscanning:
  variables:
```

SECURELOGLEVEL: "info"

> For a full list of variables available for dependency scanning, see the documentation.

1. Commit this change to the main branch, using Change log level for Python dependency scanner as a commit message. Click Commit changes.

> This commit triggers a pipeline run using your new Dependency Scanning configuration.

1. Navigate to Build > Pipelines and wait for the most recent pipeline to finish. If you want to watch its progress, go to the pipeline's details page and click on the job the gemnasium-python-dependencyscanning pill button to see the console output for the job. Remember that Dependency Scanning can take a few minutes to run.

Task E. Take Action on Vulnerabilities

> The first step in managing dependency based vulnerabilities is to verify that the vulnerability does exist in your usage of the dependency. If the vulnerability does exist in your project, you should take steps to remedy the issue.

1. Navigate to Secure > Vulnerability Report.

1. In the Activity filter, change the option to Tool, then select the option under Dependency Scanning. Notice any security vulnerabilities Dependency Scanning found in the requirements file.

1. View the vulnerability with a description of Unintended leak of Proxy-Authorization header in requests. Click the description.

1. On the description page, you will see a description of the vulnerability, including the impact of the vulnerability. You will additionally see information about the patch and workaround for the vulnerability.

1. In the top right corner, click the Status dropdown and choose Confirm.

1. In the Linked items section of the vulnerability description, click the Create issue button.

1. Assign the issue to yourself and click the Create issue button.

> Note that the description of the vulnerability was copied to the issue.

1. Navigate back to Secure > Vulnerability report. Notice that the vulnerability you just created an issue for has a link to that issue in the Activity column.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab Security Essentials Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/secessentialshandson3.md

> Estimated time to complete: 15 minutes

Objectives

Many projects depend on using containers that might contain vulnerabilities in the container image.

In this lab, you will learn how to scan for vulnerabilities in your containers.

Prerequisites

1. Open your browser to the Security Labs project that you created in Lab 1.

> If you closed the tab or lost the link, open a browser tab and start typing <https://gitlab.com/gitlab-learn-labs> in your URL, and the project should appear in your history.

1. Before beginning this lab and all later labs, you should disable any scanners that you enabled in previous labs to speed up pipeline runtime. You should have already completed this in Lab 2.

Task A. Add a Dockerfile

> A Dockerfile is a "recipe" that tells Docker how to assemble your application into a Docker image. You'll write a Dockerfile that installs your single-file Python application onto an Python 3.4 Docker image, and then packages that whole stack into a new Docker image.

1. Navigate to Code > Repository.

1. Above the repository file list, click (+) > This directory > New file.

1. In the File name field, type Dockerfile. It is important that the first letter is capitalized for industry standards.

1. The Dockerfile must specify which Linux image to install your application on. For this lab you'll use an old version of Python that has security vulnerabilities for the Container Scanner to find. Paste this into Dockerfile:

```
Dockerfile
FROM python:3.4-alpine
```

1. The Dockerfile must add your application to the Linux image specified above. Paste this at the bottom of Dockerfile:

```
Dockerfile
ADD main.py .
```

1. Your completed Dockerfile should look like this. Make any corrections necessary.

```
Dockerfile
FROM python:3.4-alpine
ADD main.py .
```

1. Add a commit message, set the target branch to main and click Commit changes.

Task B. Build the Docker image

> In this section you will define a job that builds a Docker image. To build a Docker image with a CI/CD pipeline job, you must use a GitLab Runner that's configured to use a Docker executor.

1. Navigate to Code > Repository and edit .gitlab-ci.yml.

1. Define a build stage by pasting this in your .gitlab-ci.yml, at the top of the stages list, before the -test stage. Make sure it has the same indentation as the existing - test entry beneath it:

```
yml
stages:
- build
- test
```

1. Name your new job and assign it to the build stage by pasting this at the end of .gitlab-ci.yml:

```
yml
build-and-push-docker-image:
  stage: build
```

1. Your job must run on a Docker image that contains Docker tools. This approach is sometimes called "Docker in Docker" or "dind". You'll need to specify a version of the image that we've tested and know to work well for this task. Paste this underneath the build-and-push-docker-image job that you added in the previous step:

```
yml
build-and-push-docker-image:
  stage: build
  image: docker:19.03.12
```



```
image: docker:20.10.17
```

1. Your job also needs a second Docker image that enables the Docker in Docker workflow. Specify the second image with the `services` keyword, by pasting this into your job definition:

```
yaml
build-and-push-docker-image:
  stage: build
  image: docker:20.10.17
  services:
    - docker:20.10.17-dind
```

1. It's helpful to define a variable to hold the full name and version of the Docker image you're creating, because you'll need to refer to that information more than once. You can assemble the name and version out of predefined variables that GitLab provides (remember that predefined variables generally start with `CI`). Paste this into your job definition:

```
yaml
build-and-push-docker-image:
  stage: build
  image: docker:20.10.17
  services:
    - docker:20.10.17-dind
  variables:
    IMAGE: $CI_REGISTRYIMAGE/$CI_COMMITREFSLUG:$CI_COMMITSHA
```

1. If you set a variable telling Docker not to use TLS, you won't have to worry about setting up security certificates. Add the `DOCKERTLSCERTDIR` variable.

```
yaml
build-and-push-docker-image:
  stage: build
  image: docker:20.10.17
  services:
    - docker:20.10.17-dind
  variables:
    IMAGE: $CI_REGISTRYIMAGE/$CI_COMMITREFSLUG:$CI_COMMITSHA
    DOCKERTLSCERTDIR: ""
```

1. Tell Docker to build a Docker image using the recipe in `Dockerfile`. Add the `script:` and `docker-build` lines.

```
yaml
build-and-push-docker-image:
  stage: build
  image: docker:20.10.17
```

services:

- docker:20.10.17-dind

variables:

IMAGE: \$CI_REGISTRY_IMAGE/\$CI_COMMITREFSLUG:\$CI_COMMITSHA

DOCKERTLSCERTDIR: ""

script:

- docker build --tag \$IMAGE .

Task C. Push the Docker image to Project Container Registry

> Your job needs to log in to the project's container registry so it can push your image to it. You can log in using a username, password, and registry URL that are stored in predefined variables.

1. Add the docker login line to the bottom of the script section.

yaml

build-and-push-docker-image:

stage: build

image: docker:20.10.17

services:

- docker:20.10.17-dind

variables:

IMAGE: \$CI_REGISTRY_IMAGE/\$CI_COMMITREFSLUG:\$CI_COMMITSHA

DOCKERTLSCERTDIR: ""

script:

- docker build --tag \$IMAGE .

- docker login --username \$CI_REGISTRY_USER --password \$CI_REGISTRY_PASSWORD \$CI_REGISTRY

1. Your job can push the image with a single Docker command. Add the docker push line to the bottom of the script section.

yaml

build-and-push-docker-image:

stage: build

image: docker:20.10.17

services:

- docker:20.10.17-dind

variables:

IMAGE: \$CI_REGISTRY_IMAGE/\$CI_COMMITREFSLUG:\$CI_COMMITSHA

DOCKERTLSCERTDIR: ""

script:

- docker build --tag \$IMAGE .

- docker login --username \$CI_REGISTRY_USER --password \$CI_REGISTRY_PASSWORD \$CI_REGISTRY

- docker push \$IMAGE

1. Your completed job definition should look like this. Make any corrections necessary to the

job definition in your .gitlab-ci.yml.

```
yml
build-and-push-docker-image:
  stage: build
  image: docker:20.10.17
  services:
    - docker:20.10.17-dind
  variables:
    IMAGE: $CI_REGISTRYIMAGE/$CICOMMITREFSLUG:$CICOMMITSHA
    DOCKERTLSCERTDIR: ""
  script:
    - docker build --tag $IMAGE .
    - docker login --username $CI_REGISTRYUSER --password $CI_REGISTRYPASSWORD $CI_REGISTRY
    - docker push $IMAGE
```

1. Commit the changes to the main branch with an appropriate commit message (Adding a docker file definition).

1. Navigate to Build > Pipelines to watch the progress of the new pipeline. Click on the pipeline to view the CI output for the build job.

1. When the pipeline finishes running, go to left navigation pane and click Deploy > Container Registry. Verify that your job created a new Docker image and pushed it into the project's container registry.

Task D. Enable Container Scanning

> Your application's docker image may contain known vulnerabilities. In order to prevent these vulnerabilities from reaching production, you can detect them with Container Scanning. Now that your Docker image is being built and pushed, you can enable Container Scanning.

1. Add the Container Scanning template to the existing include: section of .gitlab-ci.yml:

```
yml
include:
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/Secret-Detection.gitlab-ci.yml
  - template: Security/Dependency-Scanning.gitlab-ci.yml
  - template: Security/Container-Scanning.gitlab-ci.yml
```

> This can be added anywhere in the list of templates.

1. You will also need to tell the Container Scanning tool about which image to scan. Add the following code beneath the secretDetection job:

```
yaml
containerscanning:
```

variables:

CSIMAGE: \$CI_REGISTRY_IMAGE/\$CI_COMMITREF_SLUG:\$CI_COMMIT_SHA

1. Commit the changes with an appropriate commit message.
1. Navigate to Build > Pipelines to watch the progress of the new pipeline.
1. Open the containerscanning job to view the CI output. Wait for the pipeline to finish running.

Task E. View the results

1. Navigate to Secure > Vulnerability Report.
1. In the Tool dropdown, click Container Scanning.
1. The vulnerabilities listed are vulnerabilities detected inside of the Docker container you created. Click on any individual vulnerability to view more details

Task F. Lab Pipeline Cleanup

1. Navigate to Code > Repository.
1. Edit the .gitlab-ci.yml file.
1. Copy and paste this to overwrite all of the contents of your .gitlab-ci.yml file. This is to ensure that we do not have any unnecessary scanners running that would slow down our pipeline. We are keeping the SAST job in order to maintain the .gitlab-ci.yml file's formatting.

```
yml
stages:
- test

include:
- template: Security/SAST.gitlab-ci.yml
```

1. Set the commit message to Lab 3 pipeline reset and commit your changes to the main branch.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab Security Essentials Hands-On Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/secessentialshandson4.md

> Estimated time to complete: 15 minutes

Objectives

When you add new dependencies to a project, you need to carefully consider how the licenses of the dependencies impact the project. To ensure that license requirements are met, you can add a policy to require approval for newly detected licenses.

In addition to scanning dependencies for vulnerabilities, the dependency scanner will also record the license that each dependency uses.

The License Compliance report will generate a list of all of the licenses detected in the project that are out of compliance with the project policies.

Prerequisites

1. Open your browser to the Security Labs project that you created in Lab 1.

> If you closed the tab or lost the link, open a browser tab and start typing <https://gitlab.com/gitlab-learn-labs> in your URL if you are in Self-Paced Training, and the project should appear in your history.

1. Before beginning this lab, you should ensure that the dependency scanning template is enabled in your CI/CD pipeline. To verify this, check your `.gitlab-ci.yml` file. Under `include`, you should see the template `Security/Dependency-Scanning.gitlab-ci.yml`:

```
yml
include:
- template: Security/Dependency-Scanning.gitlab-ci.yml
```

> License scanning is completed by the dependency scanner. This template is required to generate and view license compliance reports in your project.

Task A. License Compliance Scans

1. Navigate to Secure > Dependency list.

1. Click any of the licenses to view more details about the license and the compliance requirements.

Task B. Approve and Deny Licenses

> Let's assume that your team has approved the MIT license. If any license aside from the MIT license exists in a dependency, it must be approved before the merge request can be complete.

1. Navigate to Secure > Policies.

1. Click the New policy button.

1. Click Merge request approval policy > Select policy.

1. Input any name (ex. ScanApprovedPolicy) and description for the policy.

1. Set the Policy status to Enabled.

1. In Rules, set the Select scan type dropdown menu to License Scan. Ensure that all protected branches with No exceptions is selected for the merge request target.

1. Set the Status is dropdown menu to Newly Detected.

1. Set the first dropdown in the License is section to Except.

> To exit the multi-select dropdown, click anywhere outside of it.

1. In the Select license types dropdown, click MIT License. There are several licenses with similar names, so be sure to pick the right one.

1. In the Actions section, require 1 approval from an individual user. Click your username as the approval user.

1. Leave the Override project approval settings checkboxes selected and click Configure with a merge request.

1. In the resulting merge request, click the Merge button.

1. When you create a policy, GitLab will create a new project for managing the policies. After you click Merge, you will need to navigate back to your original project.

Task C. View the License Compliance report for a pipeline

1. In the left sidebar, select Build > Pipelines.

1. Select the New pipeline button at the top right of the page.

1. Leave all of the options at their default values, then select New pipeline.

1. Monitor your pipeline and wait for each job to complete successfully.

1. Once all of the jobs are complete, refresh the page. Select the Licenses tab.

1. Notice that GitLab now describes every license except the MIT License as a policy violation. This report helps to flag existing dependencies which may contain licenses that you have not approved.

Task D. Create an MR that adds a dependency with a denied license

1. Create a new branch named add-dnspython-dependency.

1. Add a new dependency to the add-dnspython-dependency branch (not the main branch!) by pasting dnspython==2.1.0 as a new line at the end of requirements.txt.

1. Click the Create merge request button.

1. Leave all fields as their default values and click Create merge request.

1. Wait for the pipeline for the merge request to complete.

1. Once the pipeline completes, refresh the page. A license compliance scan will now display, showing 1 new license detected.

1. Return to the merge request overview.

1. The license compliance report will list all of the licenses that are no longer in compliance with the project policy. Note that the merge status will appear stating Requires 1 approval from policy.

1. A new comment is also added to the activity of the merge request, stating that a policy violation was detected.

> This merge request now requires additional approval because dnspython does not use an approved license. A reviewer can now verify if the license is ok to proceed with before the merge is complete.

Task E. Lab Policy Cleanup

> To ensure that you can merge branches into main without approval, it is best to disable the policy.

1. Navigate to Secure > Policies.

1. Click on the policy you created. Click Edit policy.

1. Click Delete Policy.

1. Click Merge on the resulting merge request.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab Security Essentials Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/secessentialshandson5.md

> Estimated time to complete: 15 minutes

Objectives

This lab demonstrates coverage-guided fuzz testing, which looks for bugs in a single function in your code. Web API fuzz testing works similarly, but is not covered here.

Fuzz testing tries to cause unexpected behaviors in your code. Any unexpected exceptions or crashes could indicate bugs or potential security issues that may need to be addressed. Fuzz testing often catches problems that other QA processes miss.

A fuzz test sends data to a function in your code. It uses program instrumentation to trace which lines of your code are exercised by that input, and uses this information to make informed decisions about how to mutate future inputs in order to exercise as much of the function's code as possible.

You must define a separate CI/CD pipeline job for each function you want to fuzz test. That said, if the function in your code-under-test calls other functions, the fuzz test will catch problems that occur anywhere in the call stack. In this lab you'll fuzz test just 1 function.

Here's a high-level description of the fuzz testing workflow in GitLab:

1. A CI/CD job runs `gitlab-cov-fuzz`, a GitLab command line utility, which in turn, runs a fuzzer. The fuzzer generates data and sends it to the fuzz target. The fuzz target sends that data to the code-under-test. The collection of all data sent to the fuzz target is called a "corpus."

1. If the code-under-test processes the data successfully (i.e., without throwing unexpected errors or crashing), the fuzzer tracks which lines of the code-under-test were exercised by that input. The fuzzer then generates mutations of the data in the corpus to exercise different parts of the code-under-test. It sends that new, mutated input to the fuzz target, repeating the cycle.

1. If any input data causes an unexpected error or crash in the code-under-test, that problem is sent back to the fuzzer, which reports it to the `gitlab-cov-fuzz` utility, which passes it to the CI/CD pipeline as a potential bug in the code-under-test. That problem then appears in the GitLab GUI, under the Security tab on the pipeline details page.

Here's a picture of the same workflow:

mermaid

flowchart TD

A(CI/CD job) <--send command to start job
return info on unhandled exceptions/crashes--> B(gitlab-cov-fuzz)

B <-- send command to run fuzzing
return info and unhandled exceptions/crashes --> C(Fuzzer)

C -- send data--> D(Fuzz Target)
D -- send data--> E(Code-Under-Test)
E -- return coverage info and info on unhandled exceptions or crashes --> C

Prerequisites

1. Open your browser to the Security Labs project that you created in Lab 1.

> If you closed the tab or lost the link, open a browser tab and start typing <https://gitlab.com/gitlab-learn-labs> in your URL if you are in Self-Paced Training, and the project should appear in your history.

1. Before beginning this lab and all later labs, you should disable any jobs and scanners that you enabled in previous labs to speed up pipeline runtime. You should have already completed this at the end of Lab 3. You do not need to disable Lab 4 License Scanning.

Task A. Write the Test Code

> Fuzz testing sends random inputs to parts of an application in an effort to cause unexpected behavior. To demonstrate this process, let's create a simple function to test.

1. This is the code that the fuzz tester will scan for bugs. Paste this Python function into a new file called `codeundertest.py` in the root of your project.

```
python
def isthirdbytezero(mybytes):
    """Return True if and only if the third byte passed in is 0."""
    return mybytes[2] == 0  start counting from 0, so "2" refers to the 3rd byte
```

> This code-under-test defines a function that expects to be passed a list of bytes. If the third byte in that list is 0, the code returns the value True.

>

> There's a bug in this code-under-test: it doesn't check to make sure you passed in at least 3 bytes. If you pass it fewer than 3 bytes, an error will occur when the code looks for, but can't find, the third byte. Different languages will do different things in this situation, but Python will throw an unexpected `IndexError`. That error could cause problems in whatever code calls this function, so this behavior is considered to be a bug. Fuzz testing is a great tool to find this bug.

1. Commit the new file with an appropriate commit message.

Task B. Write the Fuzz Target

> Fuzz testing is the only type of GitLab scanning that requires you to write code: the fuzz target. The fuzz target you see below only works with the specific code-under-test for this lab. Fuzz targets for different code-under-test would look slightly different.

1. Paste this Python fuzz target code into a new file called `FuzzTarget.py` in the root of your

project. The comments explain each line of the fuzz target code.

```
python
from codeundertest import isthirdbytezero  import function to be tested
from pythonfuzz.main import PythonFuzz      import fuzz test infrastructure
```

The fuzz engine calls a function called fuzz in the fuzz target and passes it random bytes, so we need to define a function with that name, and that function must accept 1 parameter.

```
@PythonFuzz          Python decorator required by fuzz test infrastructure
def fuzz(randombytes):  Accept random data...
    isthirdbytezero(randombytes) ...and pass it on to the code-under-test.
```

```
if name == 'main':      required by fuzz test infrastructure
    fuzz()
```

> This fuzz target is typical for Python-based fuzz testing. See the GitLab documentation for instructions on writing fuzz targets for other languages.

1. Commit the new FuzzTarget.py with an appropriate commit message.

Task C. Enable and Configure Fuzz Testing

1. Define a new stage called fuzz by pasting this line at the end of the existing stages: section of .gitlab-ci.yml. Be sure to indent it correctly.

```
yml
stages:
- test
- fuzz
```

1. Enable fuzz testing by pasting this template into the existing include: section of .gitlab-ci.yml. Be sure to indent it correctly.

```
yml
include:
- template: Coverage-Fuzzing.gitlab-ci.yml
```

1. Configure fuzz testing by defining a new job in .gitlab-ci.yml.

```
yml
fuzz-test-is-third-byte-zero:
  extends: .fuzzbase  This anchor is defined in the template included above.
  image: python:latest  This image must be able to run the code-under-test.
  script:
    Install the fuzz engine from a GitLab-hosted PyPi repo.
```

```
- pip install --extra-index-url  
https://gitlab.com/api/v4/projects/19904939/packages/pypi/simple pythonfuzz
```

Run a language-agnostic binary, specifying the type of fuzz engine,
the root of the project, and the fuzz target.

```
- ./gitlab-cov-fuzz run --engine pythonfuzz --project-path ./ -- FuzzTarget.py
```

> Fuzz test job definitions, like fuzz targets, look a little different depending on what language they're testing. See the [documentation](https://docs.gitlab.com/ee/user/applicationsecurity/coveragefuzzing/configuration) for instructions on writing fuzz test job definitions for other languages.

1. Commit the edits to .gitlab-ci.yml with an appropriate commit message.

Task D. Review the results

1. Watch the pipeline run after you've committed changes to .gitlab-ci.yml. It might take up to 3 minutes to finish.

1. When the pipeline completes, click Build > Artifacts.

1. Click on the fuzz-test-is-third-byte-zero job.

1. On this screen, you will see a summary of the fuzz job. On this screen, you can see the fuzz test that was run against the isthirdbytezero function. Notice that the output includes a bytearray index out of range error.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab Security Essentials Hands-On Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/secessentialshandson6.md

> Estimated time to complete: 15 minutes

Objectives

Compliance scans allow you to control merge requests and security scans in your instances, groups, and projects. This lab will demonstrate how to enable security and merge request policies at a group level.

Task A. Setting up a Security Policy

1. Navigate to your group.

1. In the left sidebar, select Secure > Policies.

1. Select New policy.

1. Under Scan execution policy, select Select policy.

1. For the Name, enter Group security policy.

1. Ensure the Policy status is set to Enabled.

1. For Policy Scope, apply to all projects in this group without exceptions.

1. Under Actions, set the scan to run a SAST scan. Ensure that Runner Tags is set to selected automatically and Security job template is set to default.

1. Under Conditions, select Triggers. Have the policy run for all branches with No exceptions.

1. Select Configure with a merge request.

1. Select Merge to complete your configuration.

Task B. Security Policy Scans in Action

1. Navigate to your group.

1. Select New project.

1. Select Create blank project.

1. In the Project name field input Security Compliance Testing.

1. Leave all other options as default and select Create project.

1. In the project repository, select + > New file.

1. In the Filename, input main.py.

1. Add the following code to the file:

```
py
print("Testing scanners!")
```

1. Select Commit changes.

1. In the left sidebar, select Build > Pipelines.

1. Select the pipeline that was created from your commit.

1. Review the jobs in the pipeline. Notice how a SAST scan is run on this commit even though no .gitlab-ci.yml configuration exists.

Now, the projects in your group will always have security scans, even if they aren't defined in the .gitlab-ci.yml file yet.

Task C. Merge Request Policies

With security scans in place, it would be ideal to be able to block merge requests that contain security vulnerabilities. In this section, you will create a merge request policy to achieve this goal.

1. Navigate to your group.

1. In the left sidebar, select Secure > Policies.

1. Select New policy.

1. Under Merge request approval policy, select Select policy.

1. In Name, enter the value Merge Request Security.

1. Under Policy status, ensure Enabled is selected.

1. For Policy Scope, apply the policy to all projects in this group, without exceptions.

1. Under Rules, set the scan type to Security Scan.

1. For the scanner, select All scanners.

1. Run the scan against all default branches.

1. Specify that the scan has No exceptions and finds Any vulnerabilities.

1. Set the severity level to All severity levels.

1. Set the status to New and All vulnerability states.

1. Under Actions, require 1 approval from the Owner role.

1. In Override project approval settings, make sure to uncheck the option Prevent approval by merge request's author and Prevent approval by commit author.

> We are only setting these options to off because we have one account in our group. For production environments, this is often toggled on.

1. Leave all other options as default and select Configure with a merge request.

1. Select Merge.

Task D. Merge Request Policies in Action

1. Return to your group.
1. Select your Security Compliance Testing project.
1. In the left sidebar, select Code > Branches.
1. Select New branch.
1. Name the branch testpolicy, then select Create branch.
1. Select Create merge request.
1. Uncheck Mark as draft, then select Create merge request.
1. Select Code > Open in Web IDE.
1. Open your main.py file and add the following code to it:

```
py

in = input("Enter your server ip: ")
subprocess.run(["ping", in])

print("Attempting to connect to the server")
print("Application authentication was successful")
```

1. Select the branch icon, then select Commit and push to 'testbranch'.
1. Return to your MR.
1. Allow you pipeline to complete. Once it completes, take note of the comment stating This merge request has policy violations and errors.
1. To resolve these issues, select Code > Open in Web IDE.
1. Open your main.py file and edit your main.py code to the following:

```
py
print("Attempting to connect to the server")
print("Application authentication was successful")
```

1. Select the branch icon, then select Commit and push to 'testbranch'.
1. Return to your MR.

1. Allow your pipeline to complete. Once complete, take note of the comment stating Security policy violations have been resolved.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab Security Essentials Hands-On Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/sysadminhandson.md

GitLab System Administration Labs

Lab Name	Lab Link
-----	-----
Install GitLab	Lab Link
Use GitLab Administration Commands	Lab Link
Configure GitLab Runners	Lab Link
Backup and Restore GitLab	Lab Link
Implement Sign-Up Restrictions	Lab Link
Manage GitLab Logs	Lab Link
Configure Instance Monitoring	Lab Link
Troubleshoot GitLab	Lab Link

Quick links

Here are some quick links that may be useful when reviewing this hands-on guide.

- GitLab System Administration course description

Suggestions?

If you'd like to suggest changes to the GitLab System Administration Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/sysadminhandsonlab1.md

> Estimated time to complete: 30 minutes

Objectives

In this lab, you will install GitLab and its necessary dependencies on a virtual machine using the command line.

Before you get started, complete the following:

- Open the GitLab Linux Installation page in a separate browser tab for reference.
- Open the lab setup instructions provided by the instructor to locate your assigned public IPv4 address for your Omnibus server. You will use SSH to access the training environment.

Task A. Access training environment

> Your training environment consists of two EC2 instances in AWS. You will use the first instance to deploy GitLab and the second instance to deploy a runner for your GitLab instance. For this section of the lab, use the instance designated for the GitLab installation.

1. On your local computer, open a terminal window.

1. Navigate to the directory that contains the SSH key file for your server.

1. SSH connections require that your private key file is not accessible by others. On Linux and MacOS, you can do this with the following command:

```
bash
chmod 400 <keyfilename>
```

On Windows, you can do this with the following command:

```
bash
icacls .\keyname.pem /inheritance:r
```

1. Use your assigned IP address and SSH key file to log in to the server that will host your GitLab Omnibus install:

```
bash
ssh -i <keyfilename> root@<vmipaddress>
```

>If you encounter an error like: WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!, you may need to reset your SSH known hosts. To do this, run the command `ssh-keygen -R <vmipaddress>`

1. Press <kbd>Enter</kbd>.

1. If your system displays an authentication warning, type yes and press <kbd>Enter</kbd>.

1. After typing yes, you will be connected to your server.

Task B. Install necessary dependencies

1. Before proceeding with installing dependencies, ensure that your repository is updated by

running `sudo apt-get update`.

1. Install Postfix so GitLab can send notification emails by using the command below:

```
bash
sudo apt-get install -y curl policycoreutils perl postfix
```

1. If you receive a Postfix Configuration Dialog when running this command, first select Ok by pressing the right arrow key to highlight the choice, and then pressing Enter. On the next page, select No configuration using the up and down arrow keys, then press `<kbd>Enter</kbd>`.

> For a production system, you can use this configuration to be able to configure your email settings with postfix. This will allow you to send emails from GitLab.

1. Start and enable Postfix using the `systemctl enable` and the `systemctl start` commands:

```
bash
sudo systemctl enable postfix
sudo systemctl start postfix
```

Task C. Install GitLab

1. Add the GitLab install repository via the `curl` command:

```
bash
curl https://packages.gitlab.com/install/repositories/gitlab/gitlab-ee/script.deb.sh | sudo
bash
```

1. Install the GitLab package using the command below. Use your training system's assigned public IP address in lieu of a fully qualified domain name. Make sure not to include the `<>` symbols.

```
bash
sudo EXTERNALURL="http://<yourassignedpublicip>" apt-get install -y gitlab-ee
```

> This step may take a few minutes to complete.

> For this example, we added `https://` to the front of the URL. For this set of labs, we are using the `http` protocol. If you are installing GitLab in a production environment, it is recommended to use `https://` to use the `https` protocol.

Task D. Login and reset password

1. During installation, a password is randomly generated and stored for 24 hours in `/etc/gitlab/initialrootpassword`. To view the generated password, use the command below:

```
bash
```

```
sudo cat /etc/gitlab/initialrootpassword
```

2. Copy the password shown in the output to your clipboard.
3. Using a web browser, navigate to `http://<yourassignedpublicIP>/`.
4. To log in, type in root as your username, and the password copied previously for your password.
5. Once logged in, in the upper left corner of the GitLab landing page, select your root user avatar, then Edit Profile.
6. In the left navigation pane, select Password.
7. In the Current password text box, enter the temporary root password used for initial login.
8. Complete the remaining fields with a new, permanent password of your choosing.
9. Click Save password to save the changes. You will be logged out, and will need to sign in again with your new password.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/sysadminhandsonlab2.md

> Estimated time to complete: 30 minutes

Objectives

The objective of this lab is to show various gitlab-ctl commands you can use to manage your GitLab instance by using the CLI. These commands can be run after installation. For more information, see details of maintenance commands.

Task A. Run basic service status commands

1. From a terminal prompt, SSH into your training virtual machine if not already logged in.
1. Use the gitlab-ctl maintenance command to check the status of GitLab services:

```
bash
sudo gitlab-ctl status
```

1. The output of this command will appear similar to this:

```
text
run: alertmanager: (pid 21371) 205s; run: log: (pid 21114) 254s
run: gitally: (pid 21327) 208s; run: log: (pid 20429) 399s
run: gitlab-exporter: (pid 21346) 207s; run: log: (pid 21012) 270s
run: gitlab-kas: (pid 20689) 380s; run: log: (pid 20700) 379s
run: gitlab-workhorse: (pid 21309) 208s; run: log: (pid 20851) 290s
run: logrotate: (pid 20319) 414s; run: log: (pid 20327) 413s
run: nginx: (pid 21320) 208s; run: log: (pid 20917) 282s
run: node-exporter: (pid 21338) 208s; run: log: (pid 20965) 278s
run: postgres-exporter: (pid 21380) 204s; run: log: (pid 21149) 248s
run: postgresql: (pid 20496) 391s; run: log: (pid 20517) 388s
run: prometheus: (pid 21356) 206s; run: log: (pid 21077) 258s
run: puma: (pid 20768) 303s; run: log: (pid 20781) 300s
run: redis: (pid 20365) 408s; run: log: (pid 20378) 405s
run: redis-exporter: (pid 21348) 207s; run: log: (pid 21039) 266s
run: sidekiq: (pid 20798) 297s; run: log: (pid 20806) 296s
```

> To learn more about these components and how they interact, check out the documentation.

1. In the output, you will see the pid, or process ID of each GitLab service. This process ID verifies that the process is actively running on the system.

1. Often, you may need to stop or restart a service for troubleshooting purposes. To demonstrate this process, stop the nginx service.

```
bash
sudo gitlab-ctl stop nginx
```

1. To verify that the service is down, run `sudo gitlab-ctl status`. Note the value displayed for nginx.

```
text
down: nginx: 13s, normally up; run: log: (pid 20917) 1782s
```

1. Attempt to navigate to GitLab in your web browser, or refresh the page if already there. You should see some variation of "can not connect". This is because we just turned off the web server on the GitLab instance.

1. Restart the nginx web service.

```
bash
```

```
sudo gitlab-ctl start nginx
```

1. Run `sudo gitlab-ctl status` to verify that nginx is up and running again.

text

run: nginx: (pid 22369) 7s; run: log: (pid 20917) 1852s

1. Navigate to GitLab in your web browser. The application should now load properly.

Task B. Change visibility settings

1. Log into your GitLab web instance with your root user and password from Lab 1.
1. In the bottom left corner of the main screen in the sidebar, click Admin Area.
1. You will first adjust default project visibility settings. Scroll to the bottom of the left hand navigation pane and click Settings > General.
1. Under Visibility and access controls, click Expand.
1. Change the Default project visibility to Internal by clicking the associated radio button.
1. Change the Default group visibility to Internal by clicking on the associated radio button.
1. Select Save changes at the bottom of the Visibility and access controls section to apply these changes.

Task C. Locate sign-in settings

1. The second setting we will update are Sign-In restrictions. Still under Settings > General, click Collapse next to Visibility and access controls.
1. Under Sign-in restrictions, click Expand.
1. Under Two-factor authentication, click the checkbox next to Enforce two-factor authentication.
1. Select Save changes at the end of the Sign-in restrictions section to apply this change.
1. Click Collapse next to Sign-in restrictions to close the menu.
1. After applying this change, you will be redirected to a page to setup 2FA on your administrator account. You can either enable 2FA on your account, or disable the 2FA setting to avoid this notification.

To disable the 2FA setting:

1. Select Configure it later on the 2FA page.

1. Select Admin Area in the left sidebar.
1. Select Settings > General.
1. Select Expand next to Sign-in restrictions.
1. Uncheck Enforce two-factor authentication.
1. Select Save Changes at the end of the Sign-in restrictions section

Task D. Update the header logo

You can personalize your GitLab instance by uploading a header logo.

1. On the left hand side panel, click Settings > Appearance.
1. Under the Navigation Bar section, click Choose File.
1. Select an appropriate picture from your computer to serve as a header logo, and click Open.
1. Click the Update appearance settings to